

Multivariate Time Series Classification in Resource-Constrained Embedded Systems

João Vítor C. Pessoa, Rafael S. Chaves, and Marcello L. R. de Campos

Abstract—Monitoring systems in oil wells generate large volumes of multivariate time-series data that often need to be used for local decision-making, motivating the use of embedded machine learning solutions for real-time anomaly detection at the edge. Implementing machine learning models on resource-constrained single-board computers requires efficient inference pipelines that balance accuracy, power consumption, and memory usage. This paper presents a complete pipeline for implementing quantized multilayer perceptron models on ESP32 boards for anomaly detection in oil wells. The pipeline covers data acquisition, feature extraction, model training, and post-training quantization with INT8, INT16, and INT32 representations. Experimental results show that while the unquantized model achieves 95% accuracy, the quantized versions degrade accuracy by at least 6% in the INT8 case. On the other hand, the quantized models reduce energy consumption per inference, with gains of up to 55% for INT8 and $\sim 42\%$ for both INT16 and INT32.

Keywords—neural network quantization, edge inference, oil well event detection, post-training quantization, 3W dataset

I. INTRODUCTION

The oil and gas industry constitutes one of the most critical and economically significant sectors of global energy infrastructure. Offshore and onshore well operations are inherently complex, and failures in production systems can result in severe financial losses, estimated in the millions of dollars annually, as well as catastrophic environmental accidents and risks to human life [1]. In this context, continuous and reliable monitoring of well behavior has become an operational imperative. Modern wells are equipped with dense networks of sensors that continuously acquire physical variables such as pressures, temperatures, flow rates, and valve states, producing high-frequency multivariate time series (MTS) that encode the dynamic behavior of the production system. These streams constitute the primary data source for any automated monitoring or anomaly detection system aimed at safeguarding well integrity.

The challenge of detecting and classifying anomalous events in oil and gas systems has attracted growing research attention.

João Vítor C. Pessoa and Marcello L. R. de Campos are with Programa de Engenharia Elétrica (PEE/COPPE), Universidade Federal do Rio de Janeiro (UFRJ), Rio de Janeiro, RJ, Brazil. Rafael S. Chaves is with Coordenação do Curso de Engenharia de Telecomunicações (CCGTEL), Centro Federal de Educação Tecnológica Celso Suckow da Fonseca (CEFET-RJ), Rio de Janeiro, RJ, Brazil. (email: {joao.pessoa,rafael.chaves,campos}@smt.ufrj.br).

This work was partially supported by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior-Brasil (CAPES)-Finance Code 001, by the National Institute of Science and Technology (INCT) STREAM, funded by the Brazilian National Council for Scientific and Technological Development (CNPq), process no. 409179/2024-8, and also by CNPq process nos. 312145/2023-3 and 444392/2024-6, and FAPERJ process nos. E-26/200.363/2026 and E-26/210.157/2023.

Time series forecasting has been employed to identify deviating behavior in multiphase flow meters, which share many physical characteristics with oil well production lines [2]. In addition, machine learning methods based on statistical features extracted from multivariate sensor signals have demonstrated strong performance for automatic fault detection and classification in oil wells, as shown by Marins et al. [3], who achieved 94% classification accuracy using a random forest classifier.

The sensors embedded in modern wells generate continuous multivariate streams that must be processed in near real-time. Transmitting raw high-frequency data to cloud infrastructure introduces latency, bandwidth costs, and single points of failure incompatible with production environments. This motivates deploying lightweight inference models directly on edge hardware co-located with the well. The paradigm broadly known as TinyML enables low-power single-board computers (SBCs) for the edge deployment of machine learning (ML) models [4]. ML model quantization has emerged as a central technique, converting floating-point weights and activations, which are typical to cloud-computing environments, into lower-precision integer representations more compatible with the memory and compute budgets of SBCs [5]–[8].

Despite this body of work, a critical gap remains at the intersection of quantized neural network inference and real-time industrial anomaly detection on commodity SBCs. Existing studies on quantization focus predominantly on accuracy-efficiency trade-offs in isolation, typically evaluating quantized models outside their deployment context and without covering the full chain from data preparation and model training to embedded execution [9], [10]. Furthermore, MTS data from oil wells present inherent structural difficulties, including missing variables, frozen sensors, unlabeled observations, and severe class imbalance between normal operation and rare fault events [11], that interact non-trivially with model quantization and embedded deployment. The impact of fixed-point precision choices on inference latency, energy consumption, and per-class recall has not been systematically characterized for oil wells anomaly detection, where the consequences of misclassifications carry significant operational and safety implications.

This paper addresses this gap by presenting a complete end-to-end pipeline for implementing a quantized MLP on an ESP32 board [12], with a focus on anomaly detection in oil wells using the 3W data set [13]. The pipeline spans data acquisition, feature extraction, model training, and post-training quantization (PTQ) with INT8, INT16, and INT32 representations. The specific contributions of this work are: (i)

a fully reproducible quantization pipeline for shallow MLPs on the ESP32, demonstrating that anomaly detection for oil wells can be performed directly on low-cost hardware co-located with the application; and (ii) a systematic comparison of INT8, INT16, and INT32 configurations in terms of accuracy, energy consumption, and memory usage.

II. 3W DATASET

The 3W Dataset [3], [13]–[15] was developed by Petrobras in 2017 as part of the “Specialist Alarm Monitoring” (MAE) project to advance abnormal event management (AEM) in oil wells. Version 2.0 comprises 2,228 instances sampled at 1 Hz, covering 27 process variables across 42 wells. To mitigate class imbalance, the dataset combines: (1) *Real Instances*, historical data from Petrobras plant systems preserving outliers and sensor failures; (2) *Simulated Instances*, generated with the OLGA dynamic multiphase simulator; and (3) *Hand-drawn Instances*, expert-crafted curves encoding tacit knowledge of failure signatures. Each instance is labeled at the observation level with three temporal states (normal, fault transient, steady-state fault), enabling early-detection experiments. The dataset preserves real-world issues such as missing values, frozen sensors, and unlabeled transitions.

These instances monitor eight primary process variables, including pressures and temperatures at the permanent down-hole gauge (PDG), the basic sediment and water ratio (BSW), the downhole high-integrity safety valve (DHSV), the wet Christmas tree (TPT), and the production choke valve (PCK).

For this paper, three event classes are considered: normal operation (NO), severe slugging (SS), and flow instability (FI). This selection yields a balanced three-class problem, which is suitable for evaluating the accuracy-energy trade-off on a resource-constrained SBC without the confounding effect of class imbalance. Data loading uses the ThreeWToolkit library¹, which performs automatic cleaning, temporal alignment, and class selection. The explored signal is the temperature at the top of the production pipe (T-TPT), chosen for its sensitivity to flow regime variations and relevance for the selected events [13].

III. METHODOLOGY

This section describes the procedures and design decisions that guided the experimental pipeline, structured in four stages: (i) feature extraction, (ii) model training, (iii) post-training quantization, and (iv) embedded validation.

A. Feature Extraction

For feature extraction, a sliding N -length Hann window with partial overlap between consecutive windows is used. The overlap between consecutive windows is 50%, meaning the step between the start of adjacent windows is $N/2 = 500$ time samples. Incomplete windows at the end of the series are zero-padded to preserve all available time samples.

To generate per-window labels, a Boxcar window [16] without spectral weighting is used. Within each window, the

resulting label is determined by the mode of the point-wise samples contained in the interval defined as

$$y_{\text{window}} = \text{mode}(\{y_t \mid t \in [n_0, n_0 + N]\}). \quad (1)$$

When an anomaly is present in more than half of the samples within a window, the entire segment is deemed anomalous. The strategy is robust to annotation noise and state transitions near window edges.

B. Training and Reference Model Evaluation

1) *Model Architecture*: The classifier adopted is a fully connected MLP. The choice of MLP is justified by its computational simplicity, making it suitable for deployment on resource-constrained SBCs. The MLP architecture is summarized in TABLE I.

TABLE I: Neural Network Hyperparameters.

Component	Specification / Value
Input Layer	1000 (N)
Hidden Layers	32, 16
Output Layer	3 (Logits)
Activation Function	ReLU

2) *Training and Test Subsets Split*: The dataset is divided into training and test subsets at a 8 : 2 ratio. The shuffling operates on the window indices before partitioning. The shuffle mitigates the geographic or temporal bias that would arise from a naive sequential split. Training uses the Adam optimizer [17] ($\eta = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$) with categorical cross-entropy loss, mini-batch size $B = 32$, and 100 epochs with per-epoch shuffling.

3) *Reference Model Evaluation*: The performance of the 32-bit floating-point model is evaluated on the test set in terms of classification accuracy, per-sample inference time, and power consumption. The power consumption is estimated using the CodeCarbon library², which monitors hardware during inference and estimates energy in joules and watt-hours.

C. Post-Training Quantization

Quantization is performed on the MLP weights and activation functions after training the FP32 reference model, a technique known as post-training quantization. No fine-tuning of the weights is performed during or after quantization. The integer values are derived directly from the weights and activations of the trained model. The process is repeated for three bit lengths: INT8, INT16, and INT32.

The scaling factor of each tensor is calculated using the maximum absolute value observed during calibration,

$$s_x = \frac{\max(|x|)}{2^{b-1} - 1} + \varepsilon, \quad (2)$$

where b is the target bit width and $\varepsilon = 10^{-8}$ is a regularization term to avoid division by zero. The quantized value of an element x is then obtained by

$$x_q = \text{clip}\left(\left\lceil \frac{x}{s_x} \right\rceil, -(2^{b-1}), 2^{b-1} - 1\right), \quad (3)$$

¹<https://github.com/petrobras/3W>

²<https://github.com/mlco2/codecarbon>

where $\text{clip}(a, \ell, u)$ clamps a to the interval $[\ell, u]$ and $\lceil \cdot \rceil$ rounds to the nearest integer. This scheme, called absmax symmetric quantization, ensures that the tensor dynamic range is mapped symmetrically to the representable range of the integer type, minimizing the quantization error for approximately symmetric distributions around zero, a common characteristic of neural network weights under default initialization.

Unlike weights, whose scales can be statically computed from the trained tensors, intermediate activations depend on the input distribution. For this reason, a calibration subset of up to 2000 samples randomly drawn from the training set is passed through the model in inference mode. For each layer, the maximum absolute value of post-ReLU activations is recorded across all calibration samples, and the activation scales are derived from this statistic.

The weights of each linear layer are quantized by

$$W_q^{(l)} = \text{clip}\left(\left\lceil \frac{W^{(l)}}{s_W^{(l)}} \right\rceil, -(2^{b-1}), 2^{b-1} - 1\right), \quad (4)$$

where $s_W^{(l)}$ is the scale of the l th layer for $l \in \{1, \dots, L-1\}$.

Biases are quantized by

$$b_q^{(l)} = \left\lceil \frac{b^{(l)}}{s_{\text{in}}^{(l)} s_W^{(l)}} \right\rceil, \quad (5)$$

where $s_{\text{in}}^{(l)}$ is the input scale of layer l , i.e., s_x for the first layer and the activation scales $s_h^{(l-1)}$ for subsequent layers. Biases are stored in higher-precision types than weights to preserve accumulator accuracy: `int32_t` for INT8 and INT16, and `int64_t` for INT32.

After accumulation in each layer, the accumulator is expressed in scale $s_{\text{in}}^{(l)} s_W^{(l)}$, whereas the next layer expects values in scale $s_{\text{out}}^{(l)}$. The real-valued requantization factor $M^{(l)}$ is

$$M^{(l)} = \frac{s_{\text{in}}^{(l)} s_W^{(l)}}{s_{\text{out}}^{(l)}}. \quad (6)$$

Table II summarizes the C types used in each bit-width configuration, reflecting the accumulator widths required to prevent overflow during partial sums.

TABLE II: C Types for Each Quantization Bit Length.

Bit width	Weights (W_q)	Biases (b_q)	Accumulator
8-bit	<code>int8_t</code>	<code>int32_t</code>	<code>int32_t</code>
16-bit	<code>int16_t</code>	<code>int32_t</code>	<code>int64_t</code>
32-bit	<code>int32_t</code>	<code>int64_t</code>	<code>int64_t</code>

To assess how closely each quantized model reproduces the outputs of the FP32 reference, both models are evaluated on a subset of 100 randomly selected samples from the calibration set using the following metrics.

1) *Classification Divergence*: The fraction of samples for which the class predicted by the quantized model differs from that predicted by the FP32 baseline model is given by

$$\zeta = \frac{1}{M} \sum_{i=1}^M \mathbf{1}[\arg \max(y_{\text{FP32}}^{(i)}) \neq \arg \max(y_{\text{INTn}}^{(i)})], \quad (7)$$

where M denotes the total number of evaluated dataset samples, $y_{\text{FP32}}^{(i)}$ and $y_{\text{INTn}}^{(i)}$ are the output vectors produced by the FP32 and quantized models for sample i , respectively, and $\arg \max(\cdot)$ returns the index of the predicted class. The indicator function $\mathbf{1}[\cdot]$ evaluates to 1 when the predicted classes differ and 0 otherwise.

2) *Signal-to-Noise Ratio (SNR)*: The ratio between the power of the reference signal and the power of the quantization error. The SNR in dB is given by

$$\text{SNR} = 10 \log_{10} \left(\frac{\sum_i (y_{\text{FP32}}^{(i)})^2}{\sum_i (y_{\text{FP32}}^{(i)} - \hat{y}_{\text{INTn}}^{(i)})^2} \right), \quad (8)$$

where $\hat{y}_{\text{INTn}}^{(i)} = y_q s_{\text{logits}}$ denotes the dequantized logits and s_{logits} is the scale of the output logits.

D. Export to SBC

The final stage of the pipeline embeds each quantized model into a self-contained Arduino sketch that runs on the ESP32 and has no dependencies on external machine learning libraries. The embedded validation was conducted using Wokwi [18], an online simulator for ESP32 platforms.

On the ESP32, which features a Harvard-like memory architecture with separate instruction and data buses, SRAM is the primary constraint for data. To alleviate this, arrays of weights, biases, and embedded test samples are declared using the `PROGMEM` qualifier, directing the compiler to allocate constants in Flash memory. Runtime access is performed via the appropriate read macros, whose variants are selected automatically according to the model's bit length.

Before inference, the floating-point input window must be quantized using the input scale s_x , applying the same absmax scheme. In the embedded validation, this step is performed offline: test samples are quantized in Python and embedded directly into the sketch as integer arrays in Flash memory, so that the ESP32 reads pre-quantized values and passes them to the inference function without any floating-point conversion at runtime.

Since the ESP32 lacks a hardware floating-point unit, all arithmetic must be performed in integers. Since the requantization factor $M^{(l)}$ is a real number, it must be approximated using only integer operations. The idea is to represent $M^{(l)}$ as a fraction whose denominator is a power of two by using the bit-shift given by

$$M^{(l)} \approx \frac{m}{2^{31+s}}, \quad (9)$$

where m is a 31-bit integer and $s \geq 0$ is a shift count. To find m and s , $M^{(l)}$ is left-shifted until it falls in $[0.5, 1.0)$, recording the number of shifts as s ; the result is then rounded to the nearest integer and stored as m . This guarantees that m uses all 31 bits and minimizes the approximation error. Requantization of an accumulator `acc` is then performed as two successive arithmetic right-shifts

$$\text{Req}(\text{acc}) = \text{clip}((\text{acc} \cdot m \gg 31) \gg s, \min, \max), \quad (10)$$

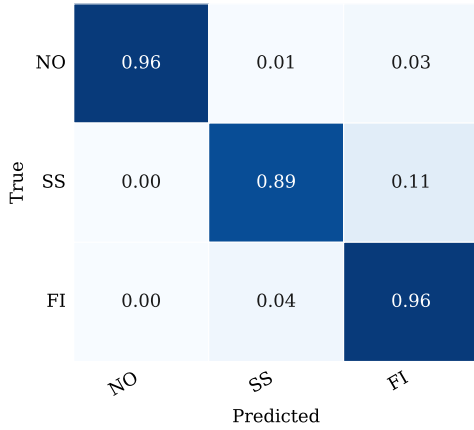


Fig. 1: Baseline normalized confusion matrix on the test set.

where $\gg$ denotes an arithmetic right-shift that preserves the sign bit. The first shift by 31 applies the fixed-point multiplication; the second shift by s completes the rescaling, with `clip` preventing overflow at the output range of the target integer type.

In the output layer, requantization is omitted, and the raw accumulators are retained, since `argmax` is invariant to scale; normalizing the logits is unnecessary for the class decision.

The embedded sketch estimates energy consumption per inference based on typical hardware constants for the ESP32 operating in active mode. The duration of each inference is measured using the `micros()` function, and the estimated per-inference energy is given by

$$E_{\text{inf}} = \underbrace{3.3}_{V_{\text{cc}}[\text{V}]} \times \underbrace{80}_{I_{\text{active}}[\text{mA}]} \times \Delta t \quad [\text{J}]. \quad (11)$$

At the end of execution, the sketch reports via the serial interface the accuracy on the embedded samples, the average inference time in μs , and the per-inference energy in joules.

IV. SIMULATION RESULTS

Each quantized model was evaluated on 100 balanced test samples divided into ten consecutive batches of ten samples each. While the FP32 baseline was evaluated on an Intel Core i7-8665U workstation at 1.90 GHz with 16 GB RAM using the CodeCarbon library, the quantized models were deployed on an ESP32 board emulated via the Wokwi online simulator [18]. All codes and the full pipeline are publicly available on GitHub.³

A. Model Performance

The FP32 baseline achieved an accuracy of 95% with a memory size of 127.5 kB, an estimated energy of ~ 7.91 mJ, and a latency of 0.505 ms per inference. The quantized models achieved an accuracy of 89%, 95%, and 95% for INT8, INT16, and INT32, respectively. In terms of energy consumption and latency, INT8 consumed 3.561 mJ with a latency of 35.611 ms

³https://github.com/joaovcpessoa/sbrt_2026_mts_classification_embedded_systems

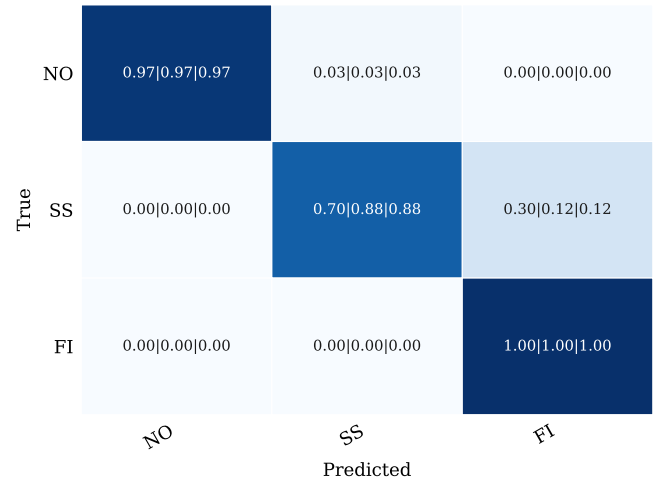


Fig. 2: Normalized confusion matrix of the quantized models on the test set. Values (left→right): INT8 / INT16 / INT32.

per inference, while INT16 and INT32 consumed 4.580 mJ and 4.589 mJ with latencies of 45.802 ms and 45.894 ms, respectively. Moreover, the quantized models have reduced memory size of 32 kB, 63.9 kB, and 127.7 kB for INT8, INT16, and INT32, respectively. Table III summarizes the quantized model results.

TABLE III: Performance and quantization comparison of the embedded models on the ESP32. Relative metrics are normalized to the FP32 baseline.

Model	Acc. (%)	Time (ms)	Energy (mJ)	Rel. Time	Rel. Energy	Div. (ζ)	SNR (dB)	Mem. (kB)
INT8	89.0	35.611	3.561	70.55×	0.45×	13.0	17.85	32.0
INT16	95.0	45.802	4.580	90.74×	0.58×	0.0	67.42	63.9
INT32	95.0	45.894	4.589	90.92×	0.58×	0.0	135.68	127.7

B. Accuracy

Figs. 1 and 2 show the normalized confusion matrix for the baseline and the quantized models with approximately 33 samples per class. In Fig. 1, the dominant error is the misclassification of 11% of SS samples as FI, which at worst would trigger the wrong corrective action. The NO class contributes a single misclassified sample, incorrectly flagged as a fault condition, which is operationally costly as it may lead to unnecessary production shutdowns. Both patterns reflect the T-TPT signal behavior, since SS and FI produce thermal disturbances of similar magnitude, and the normal operation profile can overlap with early fault transients.

In Fig. 2, for the quantized models, while FI is perfectly classified across all configurations, SS remains the dominant source of confusion: INT8 misclassifies 30% of its samples as FI, compared to 12% for INT16 and INT32. For the NO class, a single sample is misclassified across all quantized models, matching the FP32 baseline exactly and consistent with statistical variability at this sample size.

INT8 exhibits the most pronounced quantization error, with a divergence of 13% and an SNR of 17.85 dB, directly explaining the 6% accuracy drop. Despite this degradation, the model remains acceptable, suggesting that architecture refinements or quantization-aware training could potentially close this gap. INT16 and INT32 present zero divergence and fully preserve FP32-level accuracy. In terms of memory, INT32 doubles the footprint of INT16 with no measurable accuracy or divergence benefit, making the additional cost unjustifiable.

C. Latency

Table III shows that INT8 achieved the lowest inference latency, while INT16 and INT32 presented nearly identical and substantially higher values. The difference stems from the ESP32 lacking native 64-bit arithmetic support: wider weights require 64-bit accumulators that must be emulated in software via multiple 32-bit instructions, whereas INT8 accumulation fits within a single 32-bit register with no emulation overhead. It is worth highlighting that the reduced latency obtained by the baseline is due to the CPU computational power. Despite the higher latency relative to FP32 on a CPU, all quantized models remain negligible for the application, since filling the 1000-sample input window at 1 Hz requires approximately 1000 seconds of acquisition, making data capture the true system bottleneck.

D. Energy Consumption

INT8 achieves a 55% reduction in energy consumption relative to the FP32 baseline, while INT16 and INT32 yield $\sim 42\%$ savings. Their near-identical consumption reflects the shared 64-bit accumulator emulation path, whose cost dominates the inference budget regardless of weight width. The additional savings of INT8 follow directly from the absence of software emulation, where narrower accumulators translate into fewer instructions and lower active-mode energy draw.

V. CONCLUSIONS

This paper presented a complete pipeline for deploying quantized MLP models on an ESP32 for anomaly detection in oil wells, encompassing feature extraction, training, post-training quantization, and embedded execution without external machine learning libraries. Among the configurations evaluated, the INT16 stands out as the best option in terms of cost-effectiveness, preserving accuracy, achieving an energy reduction of approximately 42%, and reducing the memory by 50% compared to FP32. The INT8, although offering the greatest energy and memory savings and lowest latency, entails a 6% accuracy penalty, which may be acceptable in less security-critical scenarios or potentially mitigated through adjustments to the neural network and system parameters. Experimental results confirm the viability of low-cost embedded hardware as a practical platform for real-time industrial fault detection at the edge.

REFERENCES

- [1] Z. Wang, Y. Zhou, and T. Wang, "Dynamic Risk Assessment of Oil Spill Accident on Offshore Platform Based on the Bayesian Network," *IEEE Transactions on Engineering Management*, vol. 71, pp. 9188–9201, Nov. 2024.
- [2] T. Barbariol, D. Masiero, M. Fanan, E. Feltresi, and G. A. Susto, "Time Series Forecasting to Detect Anomalous Behavior in Multiphase Flow Meters," in *8th Forum on Research and Technologies for Society and Industry Innovation*, (Milano), pp. 54–59, IEEE, Sep. 2024.
- [3] M. Marins, B. Barros, I. Santos, D. Barrionuevo, R. Vargas, T. Prego, A. Lima, M. Campos, E. da Silva, and S. Netto, "Fault Detection and Classification in Oil Wells and Production/Service Lines Using Random Forest," *Journal of Petroleum Science and Engineering*, vol. 197, p. 107879, Sep. 2020.
- [4] H. Gauttam, G. Nain, K. Pattanaik, and P. Mendes, "Edge-AI: A Systematic Review on Architectures, Applications, and Challenges," *Journal of Network and Computer Applications*, vol. 245, p. 104375, 2026.
- [5] K. Dokic, M. Martinovic, and D. Mandusic, "Inference Speed and Quantisation of Neural Networks with TensorFlow Lite for Microcontrollers Framework," in *5th South-East Europe Design Automation, Computer Engineering, Computer Networks and Social Media Conference*, (Corfu), pp. 1–6, IEEE, Sep. 2020.
- [6] G. Ottavi, A. Garofalo, G. Tagliavini, F. Conti, L. Benini, and D. Rossi, "A Mixed-Precision RISC-V Processor for Extreme-Edge DNN Inference," in *IEEE Computer Society Annual Symposium on VLSI*, (Limasol), pp. 512–517, IEEE, Jul. 2020.
- [7] T. Flores, M. Silva, P. Andrade, J. Silva, I. Silva, E. Sisinni, P. Ferrari, and S. Rinaldi, "A TinyML Soft-Sensor for the Internet of Intelligent Vehicles," in *IEEE International Workshop on Metrology for Automotive*, (Modena), pp. 18–23, IEEE, Jul. 2022.
- [8] X. Zhao and P. Wang, "A Deployable Edge Computing Solution for Machine Condition Monitoring," in *IEEE International Instrumentation and Measurement Technology Conference*, (Glasgow), pp. 1–6, IEEE, May 2024.
- [9] Z. Zhao, K. M. Barijough, and A. Gerstlauer, "DeepThings: Distributed Adaptive Deep Learning Inference on Resource-Constrained IoT Edge Clusters," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 37, pp. 2348–2359, Oct. 2018.
- [10] M. M. H. Shuvo, S. K. Islam, J. Cheng, and B. I. Morshed, "Efficient Acceleration of Deep Learning Inference on Resource-Constrained Edge Devices: A Review," *Proceedings of the IEEE*, vol. 111, no. 1, pp. 42–91, 2023.
- [11] W. Shang, L. Ding, X. Yang, Z. Gu, and H. Sui, "Complicated Imbalanced and Overlapped Data Oversampling Approach via Hypersphere Coverage and Adaptive Differential Evolution for Anomaly Detection of Industrial Internet of Things," *IEEE Internet of Things Journal*, vol. 12, pp. 14002–14015, Jan. 2025.
- [12] E. Systems, "ESP32-DevKitC V4 Getting Started Guide," 2026. Accessed: 2026-05-18.
- [13] R. Vargas, C. Munaro, P. Ciarelli, A. Medeiros, B. Amaral, D. Barrionuevo, J. Araújo, J. Ribeiro, and L. Pierezan Magalhães, "A Realistic and Public Dataset with Rare Undesirable Real Events in Oil Wells," *Journal of Petroleum Science and Engineering*, vol. 181, p. 106223, Jul. 2019.
- [14] R. E. Vaz Vargas, A. J. de Melo Junior, C. B. de Campos Lima, J. C. D. de Araújo, M. de Araujo Fernandes, R. L. Alves Pinto, J. R. Barbosa, C. J. Munaro, P. M. Ciarelli, F. M. Varejão, F. M. Barrocas, J. N. E. Carneiro, M. L. Scramignon, R. C. Branco, T. P. Siqueira, E. T. de Lima Junior, I. de Melo Nery Oliveira, L. G. O. Lopes, L. P. de Gouveia, G. F. Peixer, and J. A. L. Cadena, "3W Dataset 2.0.0: A Realistic and Public Dataset with Rare Undesirable Real Events in Oil Wells," *Scientific Data*, Apr. 2026.
- [15] T. L. B. Dias, M. A. Marins, C. L. Pagliari, R. M. E. Barbosa, M. L. R. de Campos, E. A. B. Silva, and S. L. Netto, "Development of Oilwell Fault Classifiers Using a Wavelet-Based Multivariable Approach in a Modular Architecture," *SPE Journal*, vol. 29, pp. 4542–4556, 09 2024.
- [16] A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*. Prentice Hall, 3 ed., 2009.
- [17] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," *arXiv preprint arXiv:1412.6980*, 2014.
- [18] Wokwi, "Wokwi online esp32 and arduino simulator." <https://wokwi.com>, 2024. Accessed: 2026-05-06.